Mathematical Linkage Audit: What of Phase 1 (Mathematical Foundations) Is Actually Used in Phase 3 (System Integration)

Companion analysis for the `ugv_uav_control` package

June 28, 2026

Abstract

This document goes through the *complete* mathematics of Phase 1 (`01_Mathematical_Foundations`) and determines, object by object, how much of it is carried into Phase 3 (`03_System_Integration_ROS2`). The conclusion is sharp and important for paper writing: the link between the two phases is **architectural and conceptual, not numerical**. Phase 3 inherits the *form* of Tube MPC from Phase 1 — a nominal plan plus an ancillary error-feedback law $u = \bar{u} - Ke$, an invariant-set membership test used as a replan trigger, and the surrounding RPI / constraint-tightening vocabulary. But *none* of Phase 1's computed quantities — the LQR gain K, the Riccati matrix P, the minimal robust positively invariant (mRPI) set Ω, the tightened constraints $\bar{\mathbb{X}}, \bar{\mathbb{U}}$, the disturbance set $\mathbb{W}$, or even the plant matrices A_d, B_d — reach Phase 3. The root cause is a change of plant model (a 6-state double integrator in Phase 1 versus a 3-state single integrator in Phase 3), which makes Phase 1's artifacts dimensionally incompatible with Phase 3, compounded by hard-coded gains in the Phase 3 code. Consequently the README claim that the Full System "leverages the RPI Set and Constraint Tightening derived in Phase 1" is **not supported by the code** and should be reworded.

Contents

1 Scope and method **1**

2 The complete mathematics of Phase 1 **2**
- 2.1 Plant: discrete double integrator . . . 2
- 2.2 Feedback gain via LQR / DARE . . . 2
- 2.3 Constraints . . . 2
- 2.4 Disturbance set $\mathbb{W}$ (measurement noise through feedback) . . . 2
- 2.5 Minimal robust positively invariant (mRPI) set Ω . . . 3
- 2.6 Constraint tightening (Pontryagin difference) . . . 3
- 2.7 Nominal MPC and ancillary law (as implemented in `mpc_solvers.py`) . . . 3
- 2.8 Event-triggered variant . . . 3

3 What Phase 3 actually implements (UAV side) **3**
- 3.1 Plant: single integrator . . . 3
- 3.2 Ancillary law and ellipsoidal tube . . . 4
- 3.3 Planner . . . 4
- 3.4 Event triggers . . . 4

4 Linkage map: object by object **4**

5 Why the quantitative link is severed: the model change **5**

6 Code-level findings (relevant to honest reporting) **5**

7 Conclusions and recommendations for the paper **6**

1 Scope and method

Phase 1 is, mathematically, a single self-contained object: a robust Tube MPC design for the UAV modelled as a double integrator. It computes an LQR feedback gain, a disturbance set, an mRPI set, and tightened constraints offline (`simulations/tube_calculation.py`), saves them to `tube_data.npz`, and consumes them online in three solver classes (`lcs/mpc_solvers.py`: `StandardMPC`, `TubeMPC`, `ETTubeMPC`). It contains *no* EKF, *no* unicycle model, *no* belief-space cost, and *no* view-quality metric.

Phase 3 is the integrated ROS 2 mission. Its UAV controller (`uav_mpc_node.py` planner + `uav_ancillary_node.py` tracker) is the only part that overlaps Phase 1's domain. The EKF (`ekf_node.py`), the belief-space UGV MPC (`belief_mpc_node.py`) and the view-quality metric (`view_quality.py`) are Phase-3-native and have no Phase-1 ancestor; they are out of scope here except where noted.

For each Phase 1 mathematical object below we give the derivation as implemented, then state its fate in Phase 3 with file/line evidence.

2 The complete mathematics of Phase 1

2.1 Plant: discrete double integrator

State $x = [p^\top, v^\top]^\top \in \mathbb{R}^6$ with position $p = [p_x, p_y, p_z]^\top$ and velocity $v = [v_x, v_y, v_z]^\top$; input $u = a = [a_x, a_y, a_z]^\top \in \mathbb{R}^3$ is gravity-compensated acceleration; sample time $\Delta t = 0.05\,\text{s}$. The exact discretisation is

$$A_d = \begin{bmatrix} I_3 & \Delta t\, I_3 \\ 0 & I_3 \end{bmatrix}, \qquad B_d = \begin{bmatrix} \frac{1}{2}\Delta t^2 I_3 \\ \Delta t\, I_3 \end{bmatrix}, \qquad x_{k+1} = A_d x_k + B_d u_k + w_k. \tag{1}$$

(`tube_calculation.py:7--23`.)

2.2 Feedback gain via LQR / DARE

With $Q = \text{diag}(10, 10, 10, 1, 1, 1)$ and $R = I_3$, let $P \succ 0$ solve the discrete algebraic Riccati equation

$$A_d^\top P A_d - P - A_d^\top P B_d \,(R + B_d^\top P B_d)^{-1} B_d^\top P A_d + Q = 0, \tag{2}$$

and define the gain and closed-loop error matrix

$$K = (R + B_d^\top P B_d)^{-1} B_d^\top P A_d \in \mathbb{R}^{3\times 6}, \qquad A_K = A_d - B_d K. \tag{3}$$

A_K is Schur (spectral radius $\rho \approx 0.933$). (`tube_calculation.py:31--50`.) The same P later serves as the MPC terminal cost.

2.3 Constraints

Input box (a 30° tilt limit, $g\tan 30° \approx 5.66$, plus a vertical bound) and a state box matching the lab arena:

$$\mathbb{U}: \ |a_x|, |a_y| \le 5.66, \ |a_z| \le 4.0; \qquad \mathbb{X}: \ p_x \in [-1,1], \ p_y \in [-1.5, 1.5], \ p_z \in [0, 2.8], \ v \in [-1.5, 1.5]^3, \tag{4}$$

each converted to halfspace form $H_x x \le h_x$, $H_u u \le h_u$. (`tube_calculation.py:56--77`.)

2.4 Disturbance set $\mathbb{W}$ (measurement noise through feedback)

This is the most careful step in Phase 1. Acceleration disturbance $d \in \square([1, 1, 1.5])$ maps to state space as $w_{\text{proc}} = B_d d$. Measurement noise $\nu \in \square([0.05, 0.05, 0.1, 0, 0, 0])$ does *not* enter the plant directly; it corrupts the feedback. With the control law $u = \bar{u} - K(x + \nu - \bar{x})$ the error $e = x - \bar{x}$ obeys

$$e_{k+1} = A_K e_k + \underbrace{w_{\text{proc}} - B_d K \nu_k}_{=: w_k \in \mathbb{W}}, \tag{5}$$

so the effective disturbance set is the convex hull $\mathbb{W} = \{ w_{\text{proc}} - B_d K \nu \}$ over the vertices of both boxes (`tube_calculation.py:93--120`).

2.5 Minimal robust positively invariant (mRPI) set Ω

For (5) with A_K Schur and $w_k \in \mathbb{W}$, the mRPI set is the convergent Minkowski series

$$\Omega_\infty = \bigoplus_{i=0}^{\infty} A_K^i \, \mathbb{W}. \tag{6}$$

Phase 1 computes an outer approximation by the *support-function* method over 72 directions (axis-aligned plus pairwise diagonals in $\mathbb{R}^6$), truncated at $N = 100$ and scaled by $1/(1-\rho^{N+1})$ to remain an outer bound, giving a polytope $\Omega_H x \le \Omega_h$. (`tube_calculation.py:149--234`.)

2.6 Constraint tightening (Pontryagin difference)

The nominal trajectory is planned in the shrunken sets

$$\bar{\mathbb{X}} = \mathbb{X} \ominus \Omega, \qquad \bar{\mathbb{U}} = \mathbb{U} \ominus K\Omega, \tag{7}$$

computed by subtracting, from each halfspace right-hand side, the support of Ω (resp. $K\Omega$) in that face normal. This guarantees $x_k = \bar{p}_k + e_k \in \bar{\mathbb{X}} \oplus \Omega \subseteq \mathbb{X}$. (`tube_calculation.py:236--272`.)

2.7 Nominal MPC and ancillary law (as implemented in `mpc_solvers.py`)

`TubeMPC` ($N = 20$, $\Delta t = 0.05$) solves the quadratic program

$$\min_{\bar{X}, \bar{U}} \ \sum_{k=0}^{N-1} \|\bar{x}_k - x_{\text{ref}}\|_Q^2 + \|\bar{u}_k\|_R^2 + \|\bar{x}_N - x_{\text{ref}}\|_P^2 \tag{8}$$

subject to the nominal dynamics $\bar{x}_{k+1} = A_d \bar{x}_k + B_d \bar{u}_k$, the *tightened* constraints (7), and the **tube initial-condition constraint**

$$\Omega_H \, (x_0 - \bar{x}_0) \le \Omega_h \tag{9}$$

(solved with CVXPY/OSQP). The applied input is the ancillary law

$$u = \bar{u}_0 - K(x_0 - \bar{x}_0), \text{ clipped to } [u_{lb}, u_{ub}]. \tag{10}$$

(`mpc_solvers.py:146--273`.)

2.8 Event-triggered variant

`ETTubeMPC` re-solves only when the position error exceeds a threshold for `consecutive_threshold`= 3 steps (default trigger 0.15 m). (`mpc_solvers.py:316--349`.)

3 What Phase 3 actually implements (UAV side)

3.1 Plant: single integrator

The Phase 3 UAV is a **point-mass single integrator** with *velocity* command $u = [v_x, v_y, v_z]^\top$ and $\Delta t = 0.2\,\text{s}$ (the Gazebo velocity plugin closes the inner loops):

$$p_{k+1} = p_k + \Delta t\, u_k + w_k \quad \Longrightarrow \quad A_d = I_3, \;\; B_d = \Delta t\, I_3. \tag{11}$$

(`uav_mpc_node.py:41--42,97`.) This is a 3-state model, not the 6-state model of (1).

3.2 Ancillary law and ellipsoidal tube

`uav_ancillary_node.py` applies $u = \bar{u} - Ke$ and an *ellipsoidal* tube test

$$u_k = \bar{u}_k - Ke_k \text{ (clipped to } [-1,1]^3), \qquad \text{inside tube} \iff e_k^\top P_0\, e_k \le \alpha_k^2, \tag{12}$$

with the EKF-driven adaptive radius

$$\alpha_k = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min})\ \text{clip}\Big(\tfrac{\text{tr}\,\Sigma_{xy} - \text{tr}_{\min}}{\text{tr}_{\max} - \text{tr}_{\min}}, 0, 1\Big), \quad \alpha_{\min} = 0.5, \alpha_{\max} = 2, \text{tr}_{\min} = 0.005, \text{tr}_{\max} = 0.15. \tag{13}$$

Crucially the gains are **hard-coded**:

$$K = \text{diag}(0.6, 0.6, 0.8), \qquad P_0 = 0.186\, I_3 \tag{14}$$

(`uav_ancillary_node.py:57--58,184--191`). With these, $A_K = I_3 - \Delta t\, K = \text{diag}(0.88, 0.88, 0.84)$ is Schur — so the Schur *requirement* of Phase 1 is satisfied, but by independently chosen numbers.

3.3 Planner

`uav_mpc_node.py` ($N = 10$, $\Delta t = 0.2$) solves with SLSQP a *nonlinear* cost mixing goal tracking, an altitude regulariser, a visibility heuristic $\|p_{xy} - g_{xy}\|^2 / (z^2 + 0.1)$, and control effort — with velocity-box bounds and a linear altitude constraint $z_{\min} \le z \le z_{\max}$, but **no** terminal cost, **no** tightened constraints, and **no** tube initial-condition constraint (`uav_mpc_node.py:92--155`).

3.4 Event triggers

Replanning is governed by four prioritised conditions (tube-exit 1*a*; ArUco covariance spike 1*b*; curvature anticipation 2; drift maintenance 3) with 1 s time-lockouts, plus plan expiry — not by a consecutive-violation counter (`uav_ancillary_node.py:198--254`).

4 Linkage map: object by object

Phase 1 object	Phase 1 (math)	Phase 3 (code)	Link
Plant A_d, B_d	Double integrator, 6 states, accel input, Δt=0.05, Eq. (1)	Single integrator, 3 states, velocity input, Δt=0.2, Eq. (11)	**NOT USED**
Feedback gain K	LQR/DARE, 3×6, Eq. (3)	Hard-coded $\mathrm{diag}(0.6, 0.6, 0.8)$, 3×3	**NOT USED**
Terminal cost P	DARE solution, 6×6	none (planner has no terminal cost)	**NOT USED**
Disturbance set $\mathbb{W}$	$w_{\text{proc}} - B_d K \nu$, Eq. (5)	none; surrogate $\alpha_k(\mathrm{tr}\,\Sigma)$, Eq. (13)	**NOT USED**
mRPI set Ω	support-function polytope	ellipsoid $e^\top P_0 e \le \alpha^2$, Eq. (12)	**CONCEPT ONLY**
Constraint tightening	$\mathbb{X} \ominus \Omega$, $\mathbb{U} \ominus K\Omega$, Eq. (7)	none (raw velocity box + z bound)	**NOT USED**
Tube IC constraint	$\Omega_H(x_0 - \bar{x}_0) \le \Omega_h$, Eq. (9)	none	**NOT USED**
Nominal optimiser	QP, CVXPY/OSQP, quadratic	SLSQP, nonlinear heuristic cost	**NOT USED**
Ancillary law $u = \bar{u} - Ke$	Eq. (10)	Eq. (12), same form, diff. K/dim	**CONCEPT ONLY**
Schur stability of A_K	$\rho \approx 0.933$	$A_K = \mathrm{diag}(0.88, 0.88, 0.84)$	**CONCEPT ONLY**
Event-triggered re-solve	error threshold + counter	ellipsoid test + 4 levels + lockouts	**CONCEPT ONLY**

Summary count. Of the eleven Phase-1 mathematical objects, **zero** are used numerically in Phase 3; four cross over as *concepts/structure only* (the ancillary law form, the invariant-set membership idea, the Schur design requirement, and event-triggering); the remaining seven — including everything that constitutes the robustness *guarantee* (the mRPI set, the disturbance set, and constraint tightening) — have no Phase-3 implementation.

5 Why the quantitative link is severed: the model change

The single decisive fact is the change of plant. Phase 1 derives $K \in \mathbb{R}^{3\times6}$, $P \in \mathbb{R}^{6\times6}$, $\Omega \subset \mathbb{R}^6$ and $\bar{\mathbb{X}}, \bar{\mathbb{U}}$ for a 6-dimensional acceleration-controlled double integrator. Phase 3's UAV state is 3-dimensional and velocity-controlled. Phase 1's K multiplies a 6-vector and returns an acceleration; Phase 3 needs a map from a 3-vector error to a velocity correction. The objects are therefore **dimensionally incompatible** — Phase 1's outputs cannot be loaded into Phase 3 even in principle, regardless of code wiring. Everything quantitative follows from this: with a different A_K the mRPI set differs; without Ω there is no Pontryagin tightening; without tightening there is no recursive-feasibility guarantee to inherit.

6 Code-level findings (relevant to honest reporting)

1. **The tube-parameter load is dead code.** `uav_ancillary_node.py:47--58` attempts to load `K,P_0` from a personal absolute path (`/home/shlok-mehndiratta/.gemini/antigravity/scratch/tube_paran`) then on lines 57–58 — *outside* the `if/else` — unconditionally overwrites both with $\mathrm{diag}(0.6, 0.6, 0.8)$ and $0.186\, I_3$. Even if the file existed, the loaded values are discarded. The path is also non-portable and is *not* Phase 1's `tube_data.npz`.

2. **Phase 1's `tube_data.npz` is never referenced by Phase 3.** The only consumers of `tube_data.npz` are Phase 1's own nodes/sims; a repository-wide search finds no Phase 3 file loading it.

3. **Internal inconsistency in the theory companion.** `theory_report.pdf` (§6.1) states the fallback gain as $K = 2I_3$ giving $A_K = 0.6I_3$, and $P_0 = 6.86\, I_3$. The running code uses $K = \mathrm{diag}(0.6, 0.6, 0.8)$ ($A_K = \mathrm{diag}(0.88, 0.88, 0.84)$) and $P_0 = 0.186\, I_3$. If the report is cited in the paper, these numbers should be reconciled with the code.

4. **Altitude bound nuance.** The theory companion's deviation list says altitude limits are "declared but never enforced"; in fact `uav_mpc_node.py:137--145` builds a `LinearConstraint` enforcing $z_{\min} \le z \le z_{\max}$ in the planner. Worth correcting if that text is reused.

7 Conclusions and recommendations for the paper

1. **Do not claim Phase 3 "uses the RPI set / constraint tightening derived in Phase 1."** It does not; the robustness machinery of Phase 1 has no Phase-3 implementation, and the gains are hard-coded.

2. **Correct framing.** Phase 1 establishes the robust-MPC *formalism* (RPI set, constraint tightening, ISS of the error dynamics) that *motivates and justifies* the tube-tracking *architecture* realised in Phase 3 — i.e. theory → architecture, not theory → numbers. The genuinely shared content is the ancillary law $u = \bar{u} - Ke$, the invariant-set membership test as a replan trigger, and the Schur/event-triggering design principles.

3. **State the model change explicitly.** The double-integrator → single-integrator switch is defensible (the velocity plugin closes the inner loop) but is the reason Phase 1's numerics do not transfer; control reviewers will look for this.

4. **Locate the novelty correctly.** The real Phase-3 contributions are the Phase-3-native mathematics — the view-quality-coupled measurement noise R_k, the belief-space UGV MPC with covariance roll-out and FoV gating, and the EKF-covariance-driven adaptive tube radius α_k — none of which descend from Phase 1.